MEDICENT V3 • APLICACIÓN MÓVIL EN FLUTTER (DART)

Plan Maestro de Desarrollo Individual Paso a Paso

Modalidad: 100% Desarrollo Individual
Arquitectura: Feature-First Modular

HOJA DE RUTA AUTÓNOMA

> **Objetivo de este Plan**
> Esta guía está pensada para que construyas la aplicación móvil desde cero, paso a paso, sin abrumarte y asegurando que cada módulo se pruebe con el Backend Flask antes de pasar al siguiente. La arquitectura seleccionada es **Feature-First** (orientada por funcionalidades).

1. Explicación Profunda: ¿Qué es la carpeta `core/` y por qué existe?

En la arquitectura **Feature-First**, cada carpeta dentro de `features/` (como `auth/`, `tomas/` o `tratamientos/`) es como una "isla" independiente. Sin embargo, hay elementos que **todos los módulos necesitan por igual**. Si no existiera `core/`, tendrías que repetir código en cada feature.

`lib/core/` es el "núcleo compartido" de toda la aplicación:

Subcarpeta en `core/`	¿Qué archivos van adentro?	¿Por qué es indispensable?
`core/api/`	• `api_endpoints.dart` • `http_client.dart`	**Centraliza la comunicación:** Si mañana el backend cambia de puerto (de `5000` a `8000`) o de IP, solo editas **un solo archivo** y toda la app móvil se actualiza de inmediato. Además, inyecta automáticamente el token JWT en el encabezado `Authorization: Bearer <token>` para que no tengas que escribirlo manualmente en cada pantalla.
`core/theme/`	• `app_colors.dart` • `app_theme.dart`	**Consistencia visual y accesibilidad médica:** Define los colores de la app (ej. Verde salud, Rojo alerta de dosis omitida, Azul clínico) y fuentes legibles pensadas para adultos mayores.
`core/utils/`	• `date_formatter.dart` • `validators.dart`	**Funciones utilitarias repetitivas:** Convertir las fechas que envía Flask (ej: `"2026-11-01T08:00:00"`) a formato amigable en español (ej: "*Hoy a las 08:00 AM*"), y validar que un correo tenga arroba o que la contraseña tenga mínimo 6 dígitos.

```
// EJEMPLO VISUAL: Cómo core/api simplifica tu vida en los features:
// En lugar de escribir http://10.0.2.2:5000/api/tomas/paciente/1 a mano en cada widget:
final url = ApiEndpoints.tomasPaciente(pacienteId); // Obtenido limpiamente desde core/api
final respuesta = await HttpClient.get(url);        // Inyecta el JWT automáticamente
```

2. Plan de Ejecución Individual en 6 Fases Consecutivas

Regla de oro: Nunca empieces una fase sin haber terminado y probado la anterior.

FASE 1: El Núcleo de Conectividad (Core y Cliente HTTP)

Duración estimada: 1 Día

Meta: Lograr que Flutter pueda hablar con el Backend Flask y verificar que el servidor esté encendido.

- **Paso 1.1:** Crear `lib/core/api/api_endpoints.dart` con la constante `baseUrl` (`http://10.0.2.2:5000/api` para emulador o `http://127.0.0.1:5000/api` para emulador iOS/PC).
- **Paso 1.2:** Crear un cliente base con `http` y `flutter_secure_storage` para guardar y leer el token JWT.
- **Paso 1.3:** Probar la conexión haciendo una petición al endpoint del backend `GET /api/estado`. Si responde `{"estado": "activo"}`, el puente está 100% listo.

FASE 2: Módulo de Autenticación y Perfil (`features/auth`)

Duración estimada: 2 Días

Meta: Permitir al usuario registrarse, iniciar sesión, guardar su sesión y saber su rol (Paciente o Cuidador).

- **Paso 2.1 (Model):** Crear `features/auth/models/usuario_model.dart` con campos: `id`, `nombre`, `correo`, `rol`, `token`.
- **Paso 2.2 (Service):** Crear `features/auth/services/auth_service.dart` con métodos `login(correo, password)` y `registro(...)` consumiendo `POST /api/auth/login`.
- **Paso 2.3 (Screens):** Diseñar la pantalla `login_screen.dart` con dos campos de texto (correo y contraseña) y botón de ingreso.
- **Paso 2.4 (Flujo):** Al hacer login exitoso, guardar el token en almacenamiento seguro y navegar a la pantalla principal.

FASE 3: Dashboard Principal y Navegación (`features/home`)

Duración estimada: 1 Día

Meta: Mostrar una vista inicial personalizada según si el usuario logueado es Paciente o Cuidador.

- **Paso 3.1:** Diseñar `home_screen.dart` con un saludo (ej: "*Hola, Esteban*") y una barra de navegación inferior (BottomNavigationBar).
- **Paso 3.2:** Si el rol es `paciente`, mostrar accesos directos a "Mis Tomas de Hoy" y "Mis Tratamientos".
- **Paso 3.3:** Si el rol es `cuidador`, mostrar la lista de pacientes a su cargo consumiendo la relación `pacientes_cuidadores`.

FASE 4: El Corazón de la App: Tomas y Horarios (`features/tomas`)

Duración estimada: 3 Días

Meta: Ver la lista de tomas del día y permitir marcar una dosis como "tomada" con un solo toque.

- **Paso 4.1 (Model):** Crear `features/tomas/models/toma_model.dart` con campos: `id`, `tratamiento_id`, `fecha_hora_programada`, `estado` ('pendiente', 'tomada', 'omitida').
- **Paso 4.2 (Service):** Crear `features/tomas/services/tomas_service.dart` con métodos:
  • `obtenerTomasPaciente(int pacienteId)` consumiendo `GET /api/tomas/paciente/{id}`.
  • `marcarToma(int tomaId)` consumiendo `PATCH /api/tomas/{id}` enviando `{"estado": "tomada"}`.
- **Paso 4.3 (Screens):** Diseñar `tomas_hoy_screen.dart` con tarjetas visuales:
  • Tarjeta Verde si ya fue tomada.
  • Tarjeta Naranja si está pendiente para hoy.
  • Botón grande y claro *[Confirmar Toma]* que actualiza la pantalla al instante.

FASE 5: Módulo de Tratamientos y Medicamentos (`features/tratamientos`)

Duración estimada: 2 Días

Meta: Consultar prescripciones médicas activas y registrar nuevos tratamientos.

- **Paso 5.1 (Models):** Crear `tratamiento_model.dart` y `medicamento_model.dart`.
- **Paso 5.2 (Services):** Consumir `GET /api/medicamentos` para poblar un menú desplegable (Dropdown) con los nombres de medicamentos disponibles.
- **Paso 5.3 (Screen):** Formulario para crear tratamiento: selección de medicamento, dosis (ej: "1 cápsula"), frecuencia horaria (ej: 8 hrs), fecha inicio, fecha fin y hora inicial.
- **Paso 5.4:** Al enviar a `POST /api/tratamientos`, el backend calcula automáticamente el calendario de tomas y la app refresca la vista de tomas.

FASE 6: Pulido, Notificaciones Locales y Entrega Final

Duración estimada: 2 Días

Meta: Alarmas que suenen a la hora de la toma, manejo de errores de red y cierre de sesión seguro.

- **Paso 6.1 (Alarmas):** Integrar recordatorios locales que programen una alerta en el teléfono según la `fecha_hora_programada` de la toma.
- **Paso 6.2 (Feedback visual):** Agregar indicadores de carga (Spinners / CircularProgressIndicator) y mensajes tipo SnackBar cuando no haya internet o el backend esté apagado.
- **Paso 6.3 (Logout):** Botón en el perfil para limpiar el almacenamiento seguro (eliminar JWT) y redirigir al Login.

3. Resumen de Archivos a Crear en Cada Módulo

Módulo (`features/`)	Capa `models/`	Capa `services/`	Capa `screens/`
`auth/`	`usuario_model.dart`	`auth_service.dart`	`login_screen.dart`, `registro_screen.dart`
`home/`	-	-	`home_screen.dart`, `perfil_screen.dart`
`tomas/`	`toma_model.dart`	`tomas_service.dart`	`tomas_hoy_screen.dart`, `historial_tomas_screen.dart`
`tratamientos/`	`tratamiento_model.dart`	`tratamiento_service.dart`	`tratamientos_list_screen.dart`, `nuevo_tratamiento_screen.dart`
`medicamentos/`	`medicamento_model.dart`	`medicamento_service.dart`	`catalogo_medicamentos_screen.dart`

Checklist Diario de Autoevaluación Individual

Al final de cada jornada de programación, responde estas 3 preguntas:

1. ¿Mi nuevo archivo tiene una sola responsabilidad clara?
2. ¿Probé la pantalla con el Backend Flask corriendo (`py run.py`) y respondió los datos reales de la base de datos?
3. ¿Hice un commit en Git con un mensaje descriptivo de lo completado en el día?